

Lab Assignment 4: How to Use APIs in Python

DS 6001

Instructions

Please answer the following questions as completely as possible using text, code, and the results of code as needed. Format your answers in a Jupyter notebook. To receive full credit, make sure you address every part of the problem, and make sure your document is formatted in a clean and professional way.

Problem 0

Import the following packages:

```
In [1]: import numpy as np
import pandas as pd
import requests
import json
import os
import dotenv
import lyricsgenius
```

```
-----
ModuleNotFoundError                                Traceback (most recent call l
ast)
Cell In[1], line 7
      3 import requests
      4 import json
      5 import os
      6 import dotenv
----> 7 import lyricsgenius

ModuleNotFoundError: No module named 'lyricsgenius'
```

We will use the following user-agent:

```
In [ ]: mybot = 'ds6001'
botversion = '0.0'
botwebsite = 'Data access for a class assignment' # description becaus
email = 'smu2sr@virginia.edu' # change this to your own email, or leav
```

```
software = 'python-requests'  
softwareversion = requests.__version__  
  
useragent = f'{mybot}/{botversion} ({botwebsite}; {email}) {software}'  
useragent
```

```
Out[ ]: 'ds6001/0.0 (Data access for a class assignment; jkropko@virginia.edu)  
python-requests/2.34.2'
```

Problem 1

The hardest part of working with an API is finding and understanding the API's documentation. While the documentation (if it exists!) is essential for helping you to navigate to the data you need, they are also dense and technical.

One big reason why API documentation is difficult is that it has to speak to all use-cases. Very often, when software engineers interact with an API like genius.com's, they are building apps. Imagine that you are hired to build an events and ticketing app for a local music venue. You might want to embed a list of albums, songs, and background information in your app by connecting to the Genius API (for song lyrics and artist information) and pulling that data into your app. You might run a record label and you want to interact with this API by posting information about the musicians you've signed. Or you might want to collect data on the users of genius.com for marketing research. The documentation must provide enough information to support all of these uses. As a result, documentation tends to be very long, and most of the information you will see is neither relevant nor easy to understand. That is okay, and should be the expectation.

To effectively use API documentation, take a deep breath. Then narrow your search through the documentation to only these four things:

1. **The root:** We make requests from an API by, in part, writing out a complicated URL. In Python, we will compile the segments of this URL and let the `requests` package build the right URL. The first segment is the root, or the shared beginning of every URL associated with this API. Roots sometimes also include a version number such as `/v2` or `/v3`. Your first task is to find the root and save it as a string in Python. One complaint I have about API documentation is that sometimes the root is a bit difficult to find, and that's crazy because I think it should be written in bold letters on the homepage. Sometimes I have to look for *example calls* to the API to see the root being used.
2. **The endpoints:** The next part of an API call's URL (after the root but before

the question mark) is called the endpoint. Very commonly, APIs have many endpoints for many different types of data. Genius for example has separate endpoints for songs, artists, user accounts, its search engine, and other types of functions. Sometimes endpoints are called "resources" or something similar, but you can generally tell an endpoint because it is denoted with a slash first, such as Genius's /songs, /artists, /accounts, and /search sections under resources. Your task here is to find the endpoint or endpoints that can give you the data you need by reading their descriptions in the documentation.

3. **The endpoint parameters:** Some endpoints will also have a list of parameters that control the selection of data a request will return. There are two ways that these parameters can appear in an API call's URL. First, they can appear as "query parameters" after a question mark, using equal signs to specify parameter values, and separating parameters with & signs. If you write these parameters in a dictionary and use the `params` argument in the `requests.get()` method to supply these parameters to the API, then `requests` will format the URL properly for you. The other way parameters might appear is as "path parameters" which are part of the endpoint, sometimes surrounded by {curly braces}: this format implies that the parameter value should be part of the endpoint you supply to `requests.get()` instead of an entry in the `params` argument. Either way, your task here is to find the parameters that enable you to get the data you need and to use the correct names and values of these parameters in your Python code.
4. **How to acquire API authentication or access keys, and how to supply these credentials when interacting with the API:** It's important to remember that APIs are designed to give the data owners control over how their data is shared with users like you. There is a political dimension to that: a data owner can choose what to divulge or keep private. There's also a financial dimension: websites can charge you at varying rates for using the API, can restrict the speed and volume of your requests to prevent you from overloading systems or profiting too much off of their work, or can use a "freemium" model where only a small amount of data is free but more data carries charges. Many APIs require you to register in advance and acquire secret codes (called keys or tokens) that must be supplied back to the API to enable your use. You have two tasks here. First you need to find instructions in the documentation for acquiring API access keys, paying particular attention to whether they are asking for a credit card (Genius does not) and if so what they might charge you, and what limits exist on the amount of data

you can grab. Second, you need to find the instructions for supplying your key to the API. Sometimes that will be one of the endpoint parameters, and sometimes it will be in the headers along with your user-agent string. (When providing an API key via the headers, there's a common format that requires you to write the word "Bearer" and a space before your key. You are likely to see this format pop up sometimes.)

For the following problems, describe in words how you can get the data in question. Use the API documentation linked in each question. Be specific about

- the root,
- endpoint,
- parameters,
- method of acquiring keys,
- method for supplying these keys during an API call,
- and whether and how using the API costs money, or is limited to only certain quantities of data.

If there is more than one way to accomplish a task, describing just one of the valid approaches is fine.

If you use an LLM/gen AI to read the API documentation, prompt it to identify the root, endpoint, relevant parameters, and the mechanism for acquiring and supplying API keys. Ask for the specific parts of the documentation that supply these answers so you can double check. If instead you prompt something vague and general ("how do I access this API?") you will have a high likelihood of getting slop that either fails to work, or in the worst cases, exposes your keys or charges your credit card in ways you don't foresee.

Part a

Do not write Python code to accomplish these steps. Focus on reading the documentation and understanding what you would need to do if you did attempt to get the data. Also, do NOT give any of these websites a credit card or other payment information, and please do not copy and paste your API keys into this document.

How would you get data on the top 50 headlines from Germany today on the topic of sports? Use the News API, with documentation here:

<https://newsapi.org/docs> (Hint: here's a list of country codes: https://en.wikipedia.org/wiki/ISO_3166-1_alpha-2) [12 points]

You would set up your API key by requesting it from the website and then putting it in .env and create a user agent to use the API key. Then we would set:

```
root = https://newsapi.org/docs
```

```
endpoint = /v2/top-headlines?country=DE&category=sports&pageSize=250
```

```
params = {'api_key': facekey}
```

```
r = requests.get(root + endpoint, headers=headers, params=params)
```

Part b

Again, do not write Python code to accomplish these steps. Focus on reading the documentation and understanding what you would need to do if you did attempt to get the data. Also, do NOT give any of these websites a credit card or other payment information, and please do not copy and paste your API keys into this document.

How would you get JSON formatted data on the 100 most recent house resolutions (bills that passed a vote in the House of Representatives) in the 119th congress (elected in 2024) from the API of the U.S Congress? The documentation for this API is here: <https://api.congress.gov/> [12 points]

You would set up your API key by requesting it from the website and then putting it in .env and create a user agent to use the API key. Then we would set:

```
root = https://api.congress.gov
```

```
endpoint = /v3/bill/congress=119/billType=H&pageSize=100
```

```
params = {'api_key': facekey}
```

To view it in the form of a json data:

```
r = requests.get(root + endpoint, headers=headers, params=params)
```

```
pd.json_normalize(r.json()['results'])
```

Part c

Once more, do not write Python code to accomplish these steps. Focus on reading the documentation and understanding what you would need to do if you did attempt to get the data. Also, do NOT give any of these websites a credit card or other payment information, and please do not copy and paste your API keys

into this document.

How would you get a real-time property value estimate for a specific address, say [1 Lewis Mountain Parkway, Charlottesville, VA 22904](#) for example, using the RentCast API? The API documentation is here:

<https://developers.rentcast.io/reference/introduction> When calculating costs, suppose I want to repeat this for each of 25,000 Charlottesville area addresses.
[12 points]

You would set up your API key by requesting it from the website and then putting it in .env and create a user agent to use the API key. Then we would set:

```
root = https://developers.rentcast.io
```

```
endpoint = /reference/property-valuation/avm/value
```

```
params = {'api_key': facekey, "formattedAddress": "1 Lewis Mountain Parkway,  
Charlottesville, VA 22904"}
```

```
r = requests.get(root + endpoint, headers=headers, params=params)
```

```
pd.json_normalize(r.json()['results'])
```

To repeat this process for 25000 Charlottesville addresses:

```
root = https://developers.rentcast.io
```

```
endpoint = /reference/property-valuation/avm/value
```

```
params = {'api_key': facekey, "zipCode": "22904", pageSize=25000}
```

```
r1 = requests.get(root + endpoint, headers=headers, params=params)
```

Problem 2

Next we'll work through the steps to access the Genius API and bring data from this API into Python.

Part a

Genius.com's API is documented here: <https://docs.genius.com/#/getting-started-h1>.

Follow the same process you used in problem 1 to find the information you need to use this API. Start with learning about how to use the search endpoint to do a

search for "Bob Dylan". Write briefly your findings regarding the root, endpoint, parameters, method of acquiring keys, and the method for supplying these keys during an API call. In this case, you are specifically looking for a credential called an Access Token. This API is free and has minimal restrictions, so no need to look for that information this time.

(Hint: because you are just trying to pull data, and not set up access for additional registered users, look for "Access for Apps Without Users" in the Authentication section of the API documentation. You can find the "API Client management page" by clicking "Manage Clients" on the left.) [12 points]

```
In [6]: botname = 'ds6001'
        version = '0.0'
        website = 'Class Activity'
        email = 'smu2sr@virginia.edu'

        useragent = f'{botname}/{version} ({website}: {email}) python-requests'
        useragent
```

```
Out[6]: 'ds6001/0.0 (Class Activity: smu2sr@virginia.edu) python-requests/2.3
        4.2'
```

```
In [11]: root = 'https://api.genius.com'
        endpoint = '/search'
        params = {'q': 'Bob Dylan'}
        r = requests.get(root + endpoint, headers={'User-Agent': useragent}, p
        r
```

```
Out[11]: <Response [401]>
```

Part b

Create a `.env` file for this project. The easiest way to do this in VS Code is to click on File -> New File, then name the file ".env" exactly, with no additional characters.

Open the .env file, copy your Genius access token into it with a name, and save it. Then use Python code to load this environmental variable and save it as a Python variable. (You can print the access token to make sure it worked, but do not allow your access token to display in your final notebook). [12 points]

Part c

Using the `requests` package in Python, connect to the Genius API's /search endpoint and search for "Bob Dylan". Remember to include a dictionary under

`headers` that includes your user-agent string.

The output will be in JSON format. Genius's API is organized in a way that every individual artist has his or her own API endpoint with more detailed information about the artist. Display the portion of the JSON output that displays the API endpoint path for the data on Bob Dylan. (Hint: it is listed under `primary_artist` several branches down the JSON tree.) [14 points]

Part d

Add `/songs` to the end of the the endpoint path you found in part (c) and use this path to request the 20 most popular songs written by Bob Dylan (some of these may be performed by other artists because Dylan is one of the most frequently covered musicians of all time). Organize these data in a `pandas` data frame. [14 points]

Problem 3

One drawback of the Genius API is that it currently has no endpoint that returns song lyrics.

The `lyricsgenius` package is a wrapper around `requests` and `BeautifulSoup`, the Python package for web-scraping data off of the raw HTML of a webpage. We will cover web-scraping in module 5. But for now, follow the guide on the GitHub repository for this package (<https://github.com/johnwmillr/LyricsGenius>) for instructions on using it. Then use `lyricsgenius` to download and display the lyrics to "Tangled Up in Blue" by Bob Dylan.

Optional but recommended: call a loved one on the phone and sing "Tangled Up in Blue" to them. [12 points]